Образовательная организация _____

Кафедра _____

ОТЧЁТ ПО ПРОЕКТУ

Дисциплина «Интеграция корпоративных систем»

Интеграция TMS с WMS и ERP

Тема проекта

«Интеграция системы диспетчеризации транспорта (TMS) с WMS и ERP: проектирование сценариев „точно в срок“, SLA по времени отклика и механизмы повторных попыток»

Программный стенд LOGOS

Выполнил: _____

Группа: _____

Проверил: _____

2026

Содержание

1 Цель и требования 3
2 Архитектура и хранение данных 4
3 Контракты и последовательность обмена 5
4 Планирование доставки точно в срок 6
5 SLA и восстановление после ошибок 7
6 Интерфейс приложения 8
7 Проверка работоспособности 9
8 Демонстрация повторных попыток 10
9 Запуск и ограничения 11
Заключение и источники 12

В комплект входят исходные коды, переносная среда Python для Windows x64, инструкция запуска, презентация, сценарий защиты, протокол автоматических проверок и отдельная демонстрация интерфейса.

Основная программа выполняет обмен между четырьмя отдельными процессами. HTML-демонстрация предназначена для резервного показа и использует только заранее заданную модель в браузере.

1 Цель и требования

Цель проекта — разработать программный стенд, который согласует выполнение заказа между ERP, складской системой WMS и транспортной системой TMS. Основное внимание уделено своевременной доставке и надёжности обмена: задержка или потеря ответа одной системы не должна приводить к повторному резервированию товара либо созданию лишней перевозки.

В ERP пользователь задаёт получателя, товар, количество и окно доставки. WMS отвечает за доступный остаток, резерв, готовность и отгрузку. TMS назначает машину на плановый интервал и отслеживает перевозку. Сервис интеграции Hub управляет последовательностью команд и фиксирует результат каждого обращения.

Для учебной версии выбраны независимые имитаторы корпоративных систем. Они используют настоящие HTTP API и отдельные базы данных. Это позволяет воспроизводить недоступность сервиса, нарушение времени ответа и повторную доставку команды без подключения к коммерческим платформам.

Требование	**Реализация**
Разделение систем	Отдельные процессы ERP, WMS, TMS и Hub
Доставка в заданное окно	Расчёт плана назад от времени приёмки
Контроль времени ответа	Реальные длительности HTTP-команд и SLA
Устойчивость к сбоям	Очередь в БД, повторы и идемпотентность
Воспроизводимая защита	Управляемые отказы и ускоренные часы

Критерий успешной интеграции — согласованное завершение заказа во всех системах. Соблюдение окна доставки оценивается отдельно от успешности HTTP-запросов.

2 Архитектура и хранение данных

В проекте применяется оркестрация: Hub определяет следующий шаг, а подключённая система выполняет локальную операцию. Пользователь работает через браузер. Сервисы обмениваются JSON по HTTP на адресе 127.0.0.1. Python и FastAPI обеспечивают API, Uvicorn запускает процессы, HTTPX выполняет обращения к сервисам [1, 2].

Сервис	Порт	Зона ответственности	База
Hub	8000	План, очередь, повторы, SLA	hub.sqlite3
ERP	8001	Заказы и итоговые статусы	erp.sqlite3
WMS	8002	Остатки, резервы и отгрузки	wms.sqlite3
TMS	8003	Машины и перевозки	tms.sqlite3

Каждый сервис читает только собственную SQLite-базу. Состояние небольшого учебного стенда хранится в виде JSON-записи таблицы state. Все изменения выполняются внутри транзакции BEGIN IMMEDIATE. SQLite предоставляет механизм транзакционного изменения данных [3].

При обработке команды бизнес-изменение, ключ идемпотентности и результат сохраняются одной транзакцией. Поэтому после потери ответа сервис может вернуть прежний результат, не изменяя данные второй раз. При исключении транзакция откатывается.

Hub сохраняет этап заказа, число попыток, время следующего повтора и неизменяемое тело команды. Данные остаются на диске после завершения процесса. При повторном запуске worker продолжает незавершённые операции. Для стенда используется один worker Hub, чтобы не выполнять одну задачу конкурентно.

3 Контракты и последовательность обмена

Контракт заказа содержит UUID, получателя, код товара, количество, начало и конец окна доставки, длительности сборки, погрузки и перевозки, а также резерв времени. Модель проверяет положительное количество и корректный порядок границ окна. Время передаётся как Unix timestamp, а интерфейс отображает его в часовом поясе браузера.

Этап Hub	Получатель	Команда
reserve	WMS	Резервирование товара
book	TMS	Назначение доступной машины
plan	ERP	Подтверждение плана
ready	WMS	Готовность к отгрузке
load	WMS	Списание зарезервированного товара
depart и arrive	TMS	Отправление и прибытие
complete	ERP	Подтверждение передачи получателю

Команды передаются через POST /commands. Заголовок Idempotency-Key имеет вид order_id:stage. Тело содержит action, order_id и data. Например, для этапа load Hub отправляет действие consume; названия этапа оркестратора и команды WMS намеренно различаются.

ERP сохраняет новый заказ со статусом new. Hub опрашивает GET /outbox и импортирует такие заказы. Если Hub остановится сразу после записи заказа в ERP, опрос после восстановления всё равно обнаружит его. Повторный импорт UUID не создаёт новую задачу.

Общий статус задачи принимает значения pending, retry, dead, done или cancelled. Статус dead означает необходимость решения пользователя, а не удаление заказа. Полные схемы API доступны на /docs каждого сервиса и включены в комплект в формате OpenAPI.

4 Планирование доставки точно в срок

План строится назад от начала окна приёмки W. Из него вычитаются длительность перевозки T, погрузки L, сборки P и временной резерв B. Для расчёта все длительности приводятся к одной единице времени.

$$t_{отправления} = W - T - B$$

$$t_{готовности} = W - T - B - L$$

$$t_{начала\ сборки} = W - T - B - L - P$$

Операция	Плановое время
Начало сборки	13:10
Готовность товара и подача машины	13:40
Погрузка	13:40–14:00
Перевозка	14:00–14:50
Резерв и ожидание окна	14:50–15:00
Передача получателю	15:00 при окне 15:00–15:15

В примере сборка занимает 30 минут, погрузка — 20 минут, дорога — 50 минут, резерв — 10 минут. Если резервирование произошло поздно, готовность наступит не раньше фактического резерва плюс длительность сборки. Это время сдвигает последующие этапы и отражается в прогнозе прибытия.

Досрочное прибытие не приводит к передаче товара до начала окна. Фактическая передача считается своевременной, когда её время находится между началом и концом окна включительно. При недостатке времени интерфейс показывает риск опоздания.

Модельные часы можно ускорять и останавливать. Реальные HTTP-запросы и интервалы повторов продолжают выполняться на паузе. Это позволяет отдельно демонстрировать логистическое расписание и сетевые ошибки.

5 SLA и восстановление после ошибок

SLA в стенде задаёт норматив успешного ответа системы. Тайм-аут ограничивает длительность одной попытки. HTTPX различает типы сетевых тайм-аутов [2]; дополнительно Hub ограничивает всё ожидание команды через asyncio.wait_for. Значения выбраны для учебной демонстрации.

Система	SLA ответа	Тайм-аут попытки
ERP	1 с	3 с
WMS	2 с	5 с
TMS	2 с	5 с

Если WMS ответила за 2,5 секунды, операция успешна, но норматив 2 секунды нарушен. В показатель SLA входят только бизнес-команды Hub: число успешных попыток в пределах норматива делится на общее число попыток. Опрос состояния и первоначальное создание заказа в показатель не включены.

При сетевой ошибке, тайм-ауте или HTTP 408, 429, 500, 502, 503, 504 выполняются повторы. Всего допускаются четыре попытки. Задержки перед второй, третьей и четвёртой попытками составляют 2, 4 и 8 секунд с добавлением случайных 0–0,3 секунды. Постоянные ошибки, например нехватка товара, сразу требуют решения пользователя.

При повторе сохраняются ключ и тело команды. Получатель возвращает ранее записанный результат, а конфликтующее тело с тем же ключом отклоняет с HTTP 409. Это обеспечивает однократный бизнес-эффект при доставке команды как минимум один раз. Гарантия exactly once для сети не заявляется.

До отгрузки Hub может отменить перевозку, снять резерв и обновить ERP. Компенсация использует ту же устойчивую очередь. После отгрузки нужна отдельная процедура возврата, которая пока не реализована.

6 Интерфейс приложения

Основной экран объединяет состояние сервисов, список заказов и показатели интеграции. В карточке заказа пользователь видит окно доставки, расчётный план, фактические времена и последние попытки. Данные обновляются автоматически.

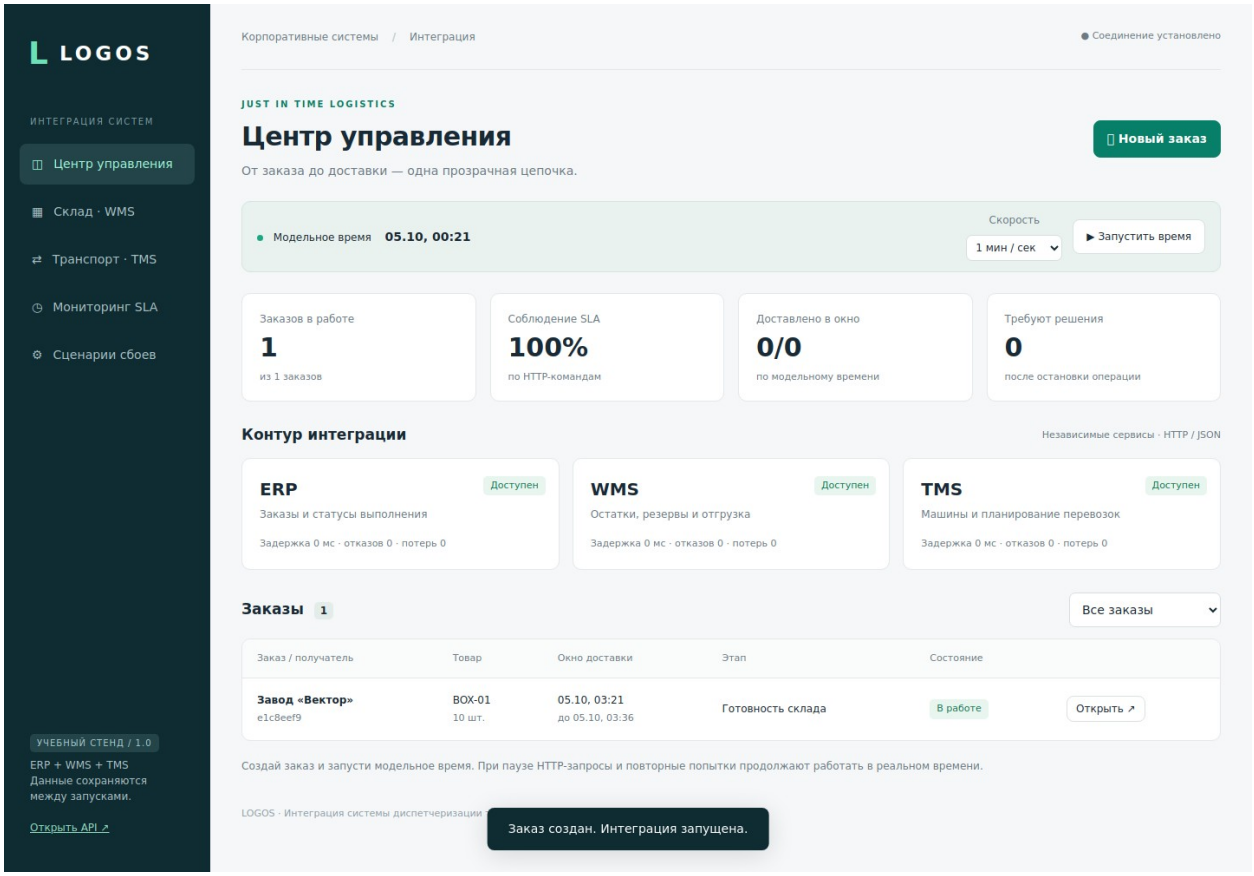


Рисунок 1 — Главный экран работающего приложения LOGOS

Раздел WMS показывает доступный остаток и резервы. Раздел TMS показывает назначенные машины и интервалы перевозок. В мониторинге выводятся HTTP-код, длительность, номер попытки и результат проверки SLA. Раздел сбоев позволяет задать задержку, количество отказов и потерю подтверждения после записи.

На рисунке показан реальный запуск серверной программы с тестовым заказом. Интерфейс не обращается к внешним ресурсам. Для документации Swagger также используются локальные файлы, что позволяет открыть её без интернета.

7 Проверка работоспособности

Сквозные тесты запускают отдельные процессы сервисов и временные SQLite-базы. Обмен идёт по настоящему HTTP. Проверяются бизнес-результаты, а не только коды ответа. Команда запуска: python -m unittest discover -s tests -v.

№	Сценарий	Результат
1	Доставка и согласованность ERP, WMS, TMS	Пройден
2	Потеря ответа TMS без второй перевозки	Пройден
3	Четыре отказа и ручное возобновление	Пройден
4	Компенсация и нехватка товара	Пройден
5	Восстановление Hub после перезапуска	Пройден
6	Успех WMS с нарушением SLA	Пройден
7	Другие данные с тем же ключом	Пройден
8	Некорректные входные данные	Пройден
9	HTTP-тайм-аут без двойного резерва	Пройден
10	Получение outbox после простоя Hub	Пройден

Все десять тестов завершились успешно в среде Linux. В браузере Chromium проверены создание заказа, открытие карточки, разделы интерфейса и журнал двух отказов TMS с успешной третьей попыткой. Ошибок JavaScript не обнаружено. На ширине 390 пикселей горизонтального переполнения страницы нет.

Во время браузерной проверки внешние сетевые обращения блокировались. Основной интерфейс и Swagger загрузились без них. Переносная среда Windows x64 включена в архив, но исполнение Windows-программы в Windows не проверялось; перед защитой нужен контрольный запуск на целевом ноутбуке.

8 Демонстрация повторных попыток

Для проверки временной недоступности TMS задано два искусственных отказа. Первые две попытки назначения машины завершились HTTP 503. Третья попытка выполнилась успешно, после чего Hub подтвердил план в ERP.

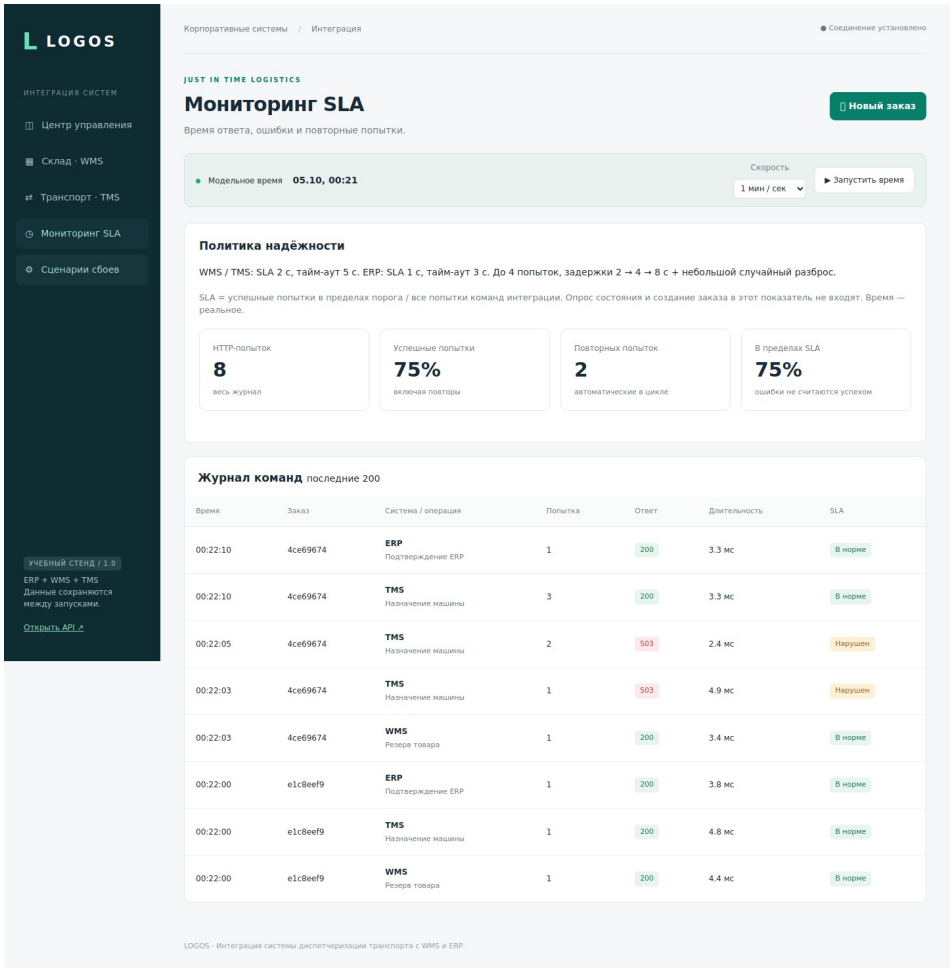


Рисунок 2 — Журнал команд с двумя отказами TMS и успешным повтором

На рисунке суммарно восемь попыток по двум заказам. Шесть попыток успешны и уложились в норматив, поэтому значение SLA составляет 75 %. Это результат конкретного демонстрационного прогона, а не оценка производительности промышленной системы.

Сценарий потери подтверждения отличается от отказа до выполнения: TMS сначала сохраняет перевозку, затем возвращает 504. Повтор с тем же ключом должен вернуть единственную существующую запись. Соответствующий сквозной тест подтверждает этот результат.

9 Запуск и ограничения

Для автономного запуска на Windows 10 или 11 x64 архив необходимо полностью распаковать в доступную для записи папку. В корне комплекта находится START_OFFLINE.bat. Он проверяет вложенный Python, запускает четыре процесса и открывает http://127.0.0.1:8000. Установка Python и подключение к интернету для этого способа не требуются.

Переносная среда подготовлена на основе официального embeddable-дистрибутива Python 3.12.10 [4]. Зависимости уже размещены рядом с ним. TEST_OFFLINE.bat запускает автоматические проверки, а папка project открывается в VS Code для просмотра и изменения кода. Для завершения работы используется Ctrl+C в терминале.

Базы создаются в project/data и сохраняются между запусками. Для новой демонстрации после остановки всех процессов папку data можно переименовать в data-backup. При следующем запуске создадутся исходные остатки. Системы слушают только локальный адрес 127.0.0.1.

В проекте пять машин и один склад. Заказ содержит одну товарную позицию. Проверка занятости машины основана на плановых интервалах. Автоматическое перепланирование других перевозок при фактическом опоздании, подбор маршрутов, грузоподъёмность и обратные рейсы пока не реализованы.

Hub обрабатывает команды последовательно, поэтому медленный ответ может задержать другие заказы. JSON-хранилище подходит для небольшого стенда, но не рассчитано на высокую нагрузку. Для развития проекта потребуются нормализованные таблицы, отдельные очереди и ограниченный параллелизм.

Файл demo/LOGOS_demo.html работает без сервера как запасной показ интерфейса. Реальную интеграцию подтверждают четыре работающих сервиса и их журнал команд.

Заключение и источники

В проекте реализован локальный стенд интеграции ERP, WMS и TMS с отдельным сервисом оркестрации. Заказ проходит от регистрации до передачи получателю. Система рассчитывает план относительно окна доставки, сохраняет фактические времена и показывает риск опоздания.

Механизм надёжности объединяет устойчивую очередь, ограниченные повторные попытки, защиту от дубликатов и компенсацию до отгрузки. Сквозные проверки подтвердили восстановление после перезапуска, обработку потерянного ответа и отсутствие повторного резервирования при тайм-ауте. Время логистической модели и время сетевых операций учитываются раздельно.

Программная реализация покрывает основные вопросы темы проекта. Дальнейшее развитие связано с реальными адаптерами ERP/WMS/TMS, многопозиционными заказами и динамическим перепланированием транспорта.

Источники

1. FastAPI. Официальная документация. https://fastapi.tiangolo.com/ (дата обращения: 05.10.2026).
2. HTTPX. Timeouts. https://www.python-httpx.org/advanced/timeouts/ (дата обращения: 05.10.2026).
3. SQLite. Transaction. https://www.sqlite.org/lang_transaction.html (дата обращения: 05.10.2026).
4. Python. Using Python on Windows. The embeddable package. https://docs.python.org/3.12/using/windows.html (дата обращения: 05.10.2026).
5. Исходные коды и протокол испытаний LOGOS 1.1. Файлы project/services, project/tests и tests/test-results.txt в составе комплекта.